Maximum Subarray Sum

→ continuous part

1	2	3	4	5

start | end (st to n-1)

0 | 0, 1, 2, 3, 4

1 | 1, 2, 3, 4

2 | 2, 3, 4

```
for(st = 0; st < n; st++) {
    for(end = st; end < n; end++) {
        ⇒ st to end
    }
}
```

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 5;
    int arr[5] = {1, 2, 3, 4, 5};

    for(int st=0; st<n; st++) {
        for(int end=st; end<n; end++) {
            for(int i=st; i<=end; i++) {
                cout << arr[i];
            }
            cout << " ";
        }
        cout << endl;
    }

    return 0;
}
```

PORTS PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL

```
apnacollege@Shradha DSAseries % g++ -std=c++11 code.cpp && ./a.out
1 12 123 1234 12345
2 23 234 2345
3 34 345
4 45
5
apnacollege@Shradha DSAseries %
```

Maximum Subarray Sum

Brute Force Approach

$O(n^2)$

{ 3, -4, 5, 4, -1, 7, -8 }

st = 0

end

CS = -1 + 5 = 4

```
maxSum
for(st = 0; st < n; st++) {
    currSum = 0
    for(end = st; end < n; end++) {
        cs += arr[end]
        maxSum = max(CS, MS);
    }
}
return MS
```

```cpp
int main() {
    int n = 5;
    int arr[5] = {1, 2, 3, 4, 5};

    int maxSum = INT_MIN;

    for(int st=0; st<n; st++) {
        int currSum = 0;
        for(int end=st; end<n; end++) {
            currSum += arr[end];
            maxSum = max(currSum, maxSum);
        }
    }

    cout << "max subarray sum = " << maxSum << endl;

    return 0;
}
```

Kadane's Algorithm

Most Optimised

{ 3, -4, 5, 4, -1, 7, -8 }

```
currSum = 0      maxSum = INT_MIN

for(i = 0; i < n; i++) {
    currSum += arr[i]
    maxSum = max(CS, MS)
    if(CS < 0)
        CS = 0
}
```

Intuition

+ve + +ve → +

-ve + +ve → +

+ve + -ve → -ve

reset to 0